

ActInf GuestStream 115.1 ~ Energy-Based Transformers and the Future of

GuestStream | Energy-Based Transformers, the Future of Scaling | 1:05:02

Hello, welcome. It is July 30th, 2025, and we're in Active Inference Guest Stream 115.1 with Alexei Gladstone, who will be presenting and discussing energy-based transformers in the future of scaling. There will be a presentation followed by a discussion, so if you're watching live, please write any questions in the live chat. Otherwise, thank you, Alexei et al., for the paper and the work and for joining, and looking forward to the presentation. Thank you. Thanks for having me, Daniel. Excited to start this presentation. So let's go ahead and get started. And I'm going to ask a philosophical question at the start of this, just to get everyone thinking. And we'll circle back to this at the end of the presentation. But that question is, would you trade off 100 times the amount of computation, so burning your GPUs for 100 times longer, for 1% better generalization, right? So generalizing 1% better out of distribution, tiny amount, right? And you might say, this question sounds absolutely ludicrous. Who in their right mind would spend 100 times the amount on computation just to do 1% better? And maybe we're already actually seeing it, right? So this is an example with opening ISO 1 and it thinking for a very long time to generalize better, albeit maybe not 1% better. But this is kind of the insight that I think will lead you into the ideas behind energy-based transformers that we are going to dive into now. So to give some contextualization, there's been lots of interest recently in what are called thinking and reasoning models broadly in AI, especially in language modeling. So in language models, particularly with large language models, we've seen tons of models. In fact, this slide is now outdated because Grok 4 and Claude 4 Sonnet have come out. So many models, including open source Chinese models, have been used to basically perform this reasoning or thinking for longer during inference. And the same thing has actually happened in Visions. There's been lots of interest in scaling diffusion models at test time as well for image generation or for video generation. So broad interest from the field in basically making models perform better at inference by kind of doing search or thinking or reasoning or test time compute, whatever you'd like to call it. But today, I'm going to take a perspective on these approaches that these approaches are ultimately unsatisfying in the quest for human level intelligence. And you might say, why? These approaches have had widespread success.

I mean, I use O1 every day and these models have been largely successful in doing great in math and coding benchmarks. So why are these approaches unsatisfying? And what is my argument today? Well, from the intellectual perspective of achieving human level intelligence, we need approaches that are modality agnostic because humans are able to reason and think over any modality. And the current approaches do not have that characteristic.

Another challenge is problem agnosticism. So current approaches very much rely on, especially in language, verifiable rewards, i.e. domains or problems such that they can be easily verified with a binary reward. An example of this is coding or math, where we can verify very easily

whether or not a coding or a math solution is correct. And ultimately, this kind of hints at another problem, which is the reliance of these approaches on external supervision. So us models, or sorry, us humans are able to get to human levels intelligence without any external rewards or verifiers or supervision. We're able to learn how to think on any modality or any problem without this type of supervision. So I'm going to make an argument today that if we want human level intelligence, right, so if we want artificial general intelligence or artificial super intelligence, whatever you'd like to call it, we really do need systems that learn to think on their own, on any modality problem, and without any additional supervision, just like us humans, right? So you might ask, okay, this makes sense. You know, I want my model to be modality agnostic, problem agnostic, supervision agnostic, etc, etc. But how can I achieve this, right? The current paradigm is very much rely on tons of humans labeling tons of data for verifiable rewards. Same thing with diffusion models, they rely on external verifiers kind of labeling how good responses are. So how can we do this without external supervision? And the point that this kind of leads to, which our paper Energy Based Transformers asks, is how can we learn to think from unsupervised learning? And this is the goal of today is to demonstrate to you that we actually can teach models to learn how to think from unsupervised learning. And there's a clever way of doing this, right? And we'll kind of go over that briefly and describe in detail how that works later on. But the general idea is in order to do thinking, we generally need a verifier. So we're going to learn to verify our predictions. And then what we'll do is we'll learn to optimize predictions with respect to this verifier, right? So given a verifier, it can tell me how good any prediction is with a given context, right? So given a sequence and a next distribution prediction for a language model, this verifier can tell me how good that distribution is. Or doing classification on ImageNet, given an image and a class, that verifier can tell me how good that class is with the image. It turns out that all these ideas can actually kind of be summed up in one framework called energy-based modeling. And in the active inference community, I'm sure many of you are familiar with this. And the broader AI community, energy-based models have been less popular. The general idea is energy-based models frame thinking or system two or reasoning as optimization over what is called a learned energy landscape. And we'll talk more in detail about what that means to train an energy-based model. So I'm going to make everyone in the next two minutes an expert. Okay, so what are feed-forward models? Feed-forward models generally take the form for supervised learning of given an input, given an X , predict a $\hat{Y}$, right? So given an image, predict the image's label, or given a sequence of tokens, predict the next token. This is the way language models work, classification models work, et cetera, et cetera. The idea behind an energy-based model is that they are given an input X and a prediction $\hat{Y}$. So for example, a context or a sequence of tokens and a distribution over next tokens. And they tell you how high energy or low energy those are. And the general idea is like low energy is high probability and high energy is low probability. So high energy is unlikely or not compatible. And then this ultimately brings about the question of, okay, I've trained this energy-based model. How do I predict using my energy-based model, right?

I have this X and this $\hat{Y}$. How do I predict this $\hat{Y}$? Because we ultimately care about using these models for prediction. It turns out that there's an elegant way to do this. That's very similar to diffusion models. The general idea is this. What we will do is we will start like diffusion models with a random prediction. So this $\hat{Y}$ will start as, for example, random noise. And what we'll do is we'll input this $\hat{Y}$ into the model and we'll get the energy of this $\hat{Y}$, right? With the context. And ideally this energy will be very high because this $\hat{Y}$ is not compatible with the current X . And then what we'll do is we'll get the gradient from this energy to this $\hat{Y}$ and iteratively refine this $\hat{Y}$ with respect to this energy. This is actually doing exactly what we said earlier, which is learning a verifier, which is this energy scalar, and then learning to optimize predictions with respect to this verifier by updating Y iteratively or updating $\hat{Y}$ iteratively. And you can kind of visualize this. So for example, if you're training a video model, the initial prediction would be a random frame. If you're training a language model, the initial prediction would be a random noise distribution. And the idea is what you'll do is you'll feed this into the model, you'll get the energy, and then you'll iteratively perform energy descent or energy minimization. And you're continuously doing that until convergence.

So I tell people about this and they often ask me, why do we care about formulating models in this way, right? Like feed forward models seem much simpler. With the feed forward model, all we do is input X and we get Y . Why do we care about doing this energy minimization thing? And there's a couple theoretical perspectives on why this is useful. The first one is flexibility. So energy-based models are very broad. It turns out that virtually anything can be, any model can be formulated as an implicit energy-based model. And you're really just doing density estimation over all the possible variables. So it's very broad. It's also very theoretically sound. So we'll kind of talk about this in more detail later. But one of the huge theoretical benefits of these energy-based models is that they could potentially be Turing complete per prediction, which means that for predictions that require an inherent amount of serial computation, energy-based models could actually solve these problems. Whereas a feed forward model cannot because feed forward models have a finite program. Energy-based models can also verify predictions with this energy scalar and can learn uncertainty in continuous state spaces pretty easily. So it's very much more difficult to do this in with feed forward models. And it turns out that energy-based models are a very broad framework. I kind of hinted at this earlier. And I like to think about energy-based models under two different perspectives, explicit energy-based models and implicit energy-based models to kind of clarify this. And the reason I say this is because a lot of models that exist, for example, GANs and diffusion models, can actually be seen as implicit energy-based models. So what this means is models where you can implicitly derive an energy function, but where this energy function is not an explicit output of the model. So just want to clarify that the models that we focus on in this paper are explicit energy-based models, where the entire model, given a set of inputs, maps to a single scalar, a scalar field called the energy. And implicit energy-based models, you need to do some work to get this scalar energy. And it's worth noting that these explicit energy-based models have never really worked well.

Right? So what I'm telling you about the attraction of these energy-based models for verification and prediction and term completeness, it sounds great, but they've never really worked. And the general intuition behind practitioners, you know, pursuing energy-based models in AI is like, don't. They don't work. They don't scale. They're unstable. There's tons of issues. They're slow, etc., etc. So the goal of this work, the TLDR, if I had to give one, would be we want to make energy-based models actually work. We want to make them relevant. We want to see how they scale. And we want to make them work practically so that we can get these great benefits of dynamic computation and verification, etc., etc., at inference. And in order to do that, we need a stable and parallelizable architecture, particularly because energy-based models have very much failed to scale because of inherent, serialness and computation and non-parallelizability, as well as instability. So when people try and scale energy-based models, they tend to be very unstable and have issues with the gradient.

And it turns out that transformers are a great fit. Transformers have amazing gradient control, having a very long residual stream. And there's strong empirical evidence suggesting that transformers can scale hugely, widely, and, you know, to multi-trillion parameter scale regimes.

Okay. So this is why we created energy-based transformers with this natural synergy. And I'd like to visualize how energy-based transformers look compared to existing approaches, just to understand, like, what are the qualitative differences? And I'd like to think about this in the perspective of some of the kind of cognitive facets or abilities we talked about earlier.

So if you have a feed-forward model, and we're just thinking about autoregressive modeling for now, the amount of computation that you have for every prediction made, i.e. with, like, current language models, is fixed. So these have finite depth, the program ends, the amount of computation it has is equivalent to the amount of depth, and that's it. You can't use any more computation for a specific prediction. RNNs generally, the way that they are trained currently, have the same flaw. So because we usually train RNNs to only update their hidden state, given new state information, they don't have this ability to dynamically compute, at least in the current regime, for most of the popular RNN variants, per prediction. Diffusion models, on the other hand, actually do have this capability. So diffusion models can have an unbounded amount of computation for every prediction being made. So that means essentially that my diffusion model can denoise for an unbounded amount of time, and therefore can be potentially turned complete. One of the challenges with diffusion models, in practice, theoretically, they have this, but in practice, they don't really, is being able to verify predictions. So let's say I generate sample A with my diffusion model, and sample B with my diffusion model. My diffusion model has no clue whether sample A or sample B is better. It generally does not work in practice. Energy-based models actually can do this. So with an energy-based model, I can generate sample A, generate sample B, pass both of these into the model, get a single scalar energy for both of them, and then use the argument of the energies to determine which sample is better.

Okay, so I'm going to dive into quickly kind of the theoretical properties that these models have. And feed-forward models, per prediction, don't really have many of the properties that would be desirable for making predictions with dynamic compute. So feed-forward models, per prediction,

are not turned complete, and they also cannot give likelihoods without tricks. So particularly in continuous state spaces, if you're using a feed-forward model, you have to resort to stuff like elbow or VQ for discretization in order to give like likelihoods, and this results in kind of some icky compression and some other undesirable traits. So generally kind of standard vanilla feed-forward models can't do many of these things in continuous state spaces. Diffusion models can be turned complete because they can have potentially unbounded amount of computation for making predictions, but in practice they don't verify or give likelihoods. Energy-based models can do all these things. So you can run energy-based inference for longer, giving you this turn completeness from a theoretical perspective, and you can also give relative likelihoods with these energy-based models. So I think it's also worthwhile to visualize how these energy-based models actually look and how they perform inference. So this is an example here of what an energy-based model would learn during training, which we call an energy landscape. And the idea is this energy landscape is a high-dimensional space spanning your predictions that tells you, it's a scalar field, telling you at each point in this prediction space how high or low energy is this, essentially how high or low probability this is, based off the prediction, its compatibility with the context. Right? So let's say we have a context of the dog-caught-the. If we feed in a random noise distribution prediction, this would likely be high energy. And the idea would be what you're doing during inference is you're performing energy minimization to get to this local minima, which corresponds to like a local energy minima that is strong and has a good prediction. This prediction is a good distribution corresponding to a token that has high likelihood under this context. And the idea is like learning this energy landscape allows you to dynamically allocate computation, verify predictions, have uncertainty in continuous state spaces, etc., etc. Dynamic computation can be performed by performing energy minimization for longer. Prediction verification can be done by getting the energy at specific points. And uncertainty can be done with highly multimodal energy landscapes, where if there's multiple local minima, you know that the model is uncertain about which prediction is best. Okay, so I'm going to dive into how energy-based transformers perform in practice.

We did a lot of scaling results in this paper to kind of dive into this and to determine whether or not they actually can scale well.

And generally, the results were very good, right? So we tested on a variety of axes. I'll show parameters scaling and flop scaling later.

But if you're just holding everything else constant, just changing the amount of data, or the batch size, or the depth, energy-based transformers scale faster than this strong transformer++ autoregressive baseline.

And it's worth noting that these are autoregressive energy-based transformers as well.

They could be bi-directional potentially, but we compared fairly to the transformer++.

And what we really care about is the slope or the asymptotic down into the right performance.

And you can see that these energy-based transformers generally have a much higher slope, so as much as like almost 36% for data scaling, and perform better down into the right.

Which is great. That's what really matters when you're scaling these models to foundation model level

compute.

We also did parameter scaling, or these are commonly referred to as scaling laws in NLP.

And you can see that the trend for energy-based transformers is generally better, especially in video, for basically scaling during pre-training.

We also did downstream inference results, or what I like to call thinking scaling, and basically testing the generalization of these models given a similar pre-training performance. And you can see that at first the energy-based transformers have more perplexity degradation, which corresponds to performing worse.

But when you basically run these energy-based transformers for longer, or have them think for longer, the performance actually gets better.

So asymptotically, they generalize much better than the transformer++, by dynamically allocating more computation.

And then another thing that's great,

is like we see many of the metrics that we care about with these energy-based transformers improving during training.

So verification improves as you train the model for longer, which essentially means that if you were to train a foundation model with energy-based transformers, the verification would get better and better during training.

which is great for avoiding adversarial examples, or generalizing even better during test time.

We also compared energy-based transformers to diffusion models, and this were bidirectional variants, on like a simple image denoising task.

And we can see that energy-based transformers generally denoise much faster, i.e. with less forward passes, and have better quality.

So the downstream metrics are better.

And one thing that's really important, that I think is potentially very impactful, is representation learning.

And representation learning with diffusion models has tended to become, tend to be very difficult.

And there are a lot of tricks that you need for representation learning with diffusion models.

With energy-based transformers, we've pretty easily seen that they learn decent representations, albeit better than diffusion models.

One other great property that we kind of investigated qualitatively,
was the ability to learn uncertainty.

And the idea is like,

and you could do this in continuous space as well,
we want the models to know what they don't know.

And this is a huge challenge,
and perhaps like why a lot of the current models struggle
at inference,
and with hallucination,
and other stuff like this.

Because they don't really know what they don't know.

So generalizing out of distribution
tends to require knowing what you don't know.

And we can see here
that energy-based transformers
have higher energy for tokens that are hard to predict.

So for example, a quick brown fox.

Same thing with research and problem here.

Whereas tokens that are easier to predict,
such as is, a, but, period,
have lower energy.

So this is very promising
for principal generalization out of distribution,
or for teaching models what they don't know.

And that is one of the general
kind of theoretical benefits of energy-based models,
especially in continuous state spaces.

Okay.

This wraps up the last of the results.

Now I'm going to transition the talk a bit,
and talk a bit about something more reflective,
and sort of prospective,
based off a lot of the learnings from this paper.

Right?

So I conducted a lot of experimentation for this paper,
lots of scaling runs,
and I kind of realized something.

Realized that there's something deeper going on.

And particularly,
something that is kind of,
the whole field is trending towards.
And that's what I'm going to talk about.

So if we ask the question,
where are we headed?

Right?

Where are models headed?

As we train them longer and longer.
And there's a trend in all loss plots.

So if we can imagine
that we have 10,000 times the amount of computation
that we have now,
virtually all models would approach
approximately zero loss,
or close to zero loss.

Right?

So given 10,000 times the amount of GPUs,
instead of 10,000 GPUs having,
I don't know, 100 billion or whatever.

All models will approach zero loss.

And this kind of looks like this.

So all models eventually achieve close to zero loss,
or whatever the irreducible slash random noise
or loss corresponding to prediction of random noise is.

And I kind of like to visualize that
as a perplexity of one,
or an MSE of 0.5 here.

Right?

And in fact,

I'd argue that frontier labs
are already starting to achieve this.

Right?

So frontier labs are already very much saying
that they are not compute bound,
but rather that they are data bound.

That data is the main issue.

And what this kind of signals

is most of these models
are achieving close to zero loss.

Right?

Or potentially what is called
the irreducible loss.

And what this means is that
we care much less
when we're not compute bound,
but rather data bound
about convergence speed.

Right?

So all of these models,
regardless of the scaling rate,
even if EBT's do scale faster
during pre-training,
reach the same kind of zero loss
or close to zero loss.

Right?

So different journeys,
but the same destination.

Okay.

So what am I getting to?

Right?

This is like approximate zero loss,
irreducible,
the best models can possibly do,
et cetera, et cetera.

What matters here?

Right?

If models really are approaching
this kind of close to zero loss regime,
what actually matters?

And I kind of hinted at this earlier,
but it's not the scaling rate.

So all models will eventually
kind of reach close to zero loss,
all decent models at least.

So the question is like,

what matters here?

And you might say,

what about inference speed

or flops or latency?

And I would also argue

that these don't matter.

If we have 10,000 times

the amount of computation,

models are already getting close

to being able to run on device

with 10,000 times the amount of compute,

inference speed and flops

won't really matter.

So is our job done?

Right?

Do we have to do anything else

in this 10,000 X compute regime

once our models achieve close to zero loss?

And today I'm going to argue

that the answer is no

and that we have a lot of work left to do.

And this is the insight that I gained

from doing this project, right?

So there's sort of a secret,

an unspoken secret of AI research

that a lot of the big tech companies

don't want the public to know,

which is that zero training loss

does not equal real world perfection

or 100% benchmark saturation

does not equal real world perfection.

In other words,

even if your model achieves 100%

on every single difficult AI benchmark

there is

and does perfect on your validation set,

this model will still not generalize

perfectly out of distribution.

Right?

So in other words,
there will always,
always, always be novel,
unseen situations
in the real world
that your model has to adapt to
and that it has to generalize too.

And it turns out that
humans adapting to this novelty
is what we are very good at.

Right?

So we're able to generalize every day
to out of distribution scenarios
that we have never seen in our life.

In fact,

I really like this example
of a counterintuitive fact
about just how much we generalize.

So if you just think hypothetically
for a second

about training a model
on all possible human data,
so all video data
that humans have ever seen,
all language data
that humans have ever spoke,
all audio data,
et cetera, et cetera.

Right?

The crazy thing about this data
is that most of it
will still be out of distribution.

Most of this data
will still be
very much data
that the model
has to generalize to.

In fact,
something very interesting
is that
you can mathematically guarantee
that this video
that you have observed
for the past five minutes,
this is the first time
you are seeing in this video.

In fact,
this is the first time
anyone in our universe
has ever observed
this past five minute video
that you're observing.

Right?

So this is how high dimensional
the data we are dealing with is.

And that's why
this out of distribution
generalization problem
is the core problem
that we have with AI right now.

And that we'll have to tackle
over the next 10 years.

So what matters
in the zero loss regime,
I kind of said this earlier,
is generalization.

Right?

So how your model performs
on this out of distribution data.

And I like to refer to this
as the era of generalization.

Right?

So this era
where all models
in the frontier labs

are essentially achieving
the same performance
or approximately zero loss.

What really matters
is how well
does your model generalize?

And this kind of explains
why there's been
a lot of interest
in reasoning
or thinking models
because these models
allow for better generalization
at test time.

More adaptability,
more test time compute,
more thinking,
et cetera, et cetera.

Okay.

This brings us to the question,
which is
if we care about generalization
instead of just flop scaling,
how do we measure it?

Right?

And today I'm going to propose
a simple metric
that I believe
will be more helpful
than relying
on many downstream benchmarks.
benchmarks.

And that is because
downstream benchmarks
tend to be very high
bias and variance.

Right?

So depending on which

benchmarks you use
to measure models,
they'll look better or worse.

You can see this
in the frontier.

You know,
the benchmarks
that different companies use
show their bench,
their models being better,
but oftentimes
they're worse,
et cetera, et cetera.

So I'm going to even
insight into a potentially
more useful
and practical metric
for generalization,
which is data efficiency.

Right?

And I'll be arguing
that data efficiency
is actually a form
of generalization.

And the reason for this
is if I have given you
three concepts,
sorry,
three examples,
and you were able
to learn a new concept
given those three examples,
but it takes me
four examples
to learn such a new concept,
you have generalized better.

Right?

So you have used

less data
to achieve
the same compression rate
or the same representations
or potentially better representations
with less data.

So that's data efficiency,
but you've also generalized better.

So given that you keep
everything constant,
same metric,
same architecture,
same parameter count
because you need
the same compression ratio,
if you achieve
the same metric
during pre-training
with less data,
your model is actually
generalizing better.

And that's what I'll argue
for here today.

Okay.

So I've claimed
that data efficiency
is how we will measure
generalization
and that generalization
is what we should care about.

So this ultimately brings us
as researchers
or engineers
to the question
of how do we measure models?

Right?

What X axis
do we use

to measure model performance?

And over the last five years,

it's an X axis

that makes energy-based

transformers look very bad,

which is flop scaling.

Energy-based transformers

needing to do MCMC

or optimization

during pre-training

are inherently more expensive,

although I'm sure

you can make this better,

but are inherently

more expensive

than feed-forward models,

right?

And hence why you see

that this flop count

is much higher.

So you might say,

okay,

these models are terrible

for flop scaling.

What about the error

of generalization?

Right?

So what about

when all we really care about

is generalization?

If all models

achieve close to the zero loss

or close to approximately

zero loss,

then what really matters

is not their flop scaling

because all models

will eventually achieve this,

but actually their data efficiency
and their generalization.

And I like to propose
that we take a look
at this graph
and focus more on this
over the next couple of years
because generalization
is becoming the big bottleneck.
Okay.

Now I'm going to bring the talk
back to a question
from the start,
which is,
would you trade off
100X times
the amount of compute
for 1% better generalization?

So running your GPUs
for 100 times longer.

And I hope that
under this perspective,
under this perspective
that potentially we are
in this era of generalization
where the bottleneck
really is just generalizing
out of distribution
rather than being
more flop efficient,
the answer
becomes an undoubtable yes
because generalization
will be all that we care about.

That's it for the presentation.

I'm happy to answer
any questions
and thank you very much

for listening.

Awesome.

Great talk.

Okay.

While everyone in the live chat

gets some questions

ready and enters them,

which I will read to you,

could you just give

a little background?

How did you come

to this problem?

What about grad school?

What was your trajectory

to get to this question

and collaboration?

Yeah, good question.

I would say that

I very much started

working on energy-based models

just from a,

I thought they were

very intuitive perspective.

So to me,

you know,

if you ask me,

like,

is the brain doing

something more similar

to energy-based modeling

or feed-forward modeling?

And no doubt,

there's no doubt in my mind

that it's more similar

to energy-based modeling,

at least in some respects,

particularly with regards

to like some of the capabilities

we talked about, right?

So being able to dynamically
allocate computation,
verify predictions.

But I think one key thing
is also learning like this
joint distribution over variables.

I think that's very important,
especially when we will
eventually take these
to multimodal learning
to kind of learn
like this compatibility
between all the input variables
and like predictions, right?

So you don't just learn
given an image
to map to a classification head.

It's not like there's
some classification head
in my brain
that tells me
the logic value
for each class
when you're doing
image classification.

No, it doesn't work like that,
right?

You learn this joint distribution
over all the possible images
in all the possible language
and then you learn
how compatible these are, right?

You know whether this image
of a dog
is compatible
with the word dog.

You don't just learn

a distribution
over the possible classes.
So to me,
they were always very intuitive
and that's kind of
why I started working on them.
And then I saw that
there was lots of room
for reasoning with them
and eventually
started working on them
during around two years ago
and yeah,
I just kept working on them
for a while
and making them,
trying making them work
and exploring
and yeah,
now we're here.
So that's the journey.
Very much just curiosity-led.
Started from the energy maxima,
now we're here.
Yeah.
Okay.
There's so many entry points
and I'm sure people
will have interesting questions,
but you brought up
one of the advantages
being the flexibility.
flexibility.
So what exactly
is that energy
landscape's flexibility?
What data matters
and why?

What could we say
about what kinds
of generalization
would be
in
out of distribution scope
versus out of
out of distribution scope?

Like,
what is it about
the energy landscape?

Do you just dump
random data?

Is it about
organic data?

What is it that
is about the data
and what does
the energy landscape
reflect

that
would or
does have
flexibility?

Yeah.

Um,
this is a challenging
question.

I would say,
uh,
the,
the general intuition
is like this energy
landscape is learned
over all possible
values,
right?

Based off the data.

So your data shapes
this landscape
that's learned
over all possible
prediction values
and like a feedforward
model doesn't learn
this kind of like,
uh,
landscape over all
possible values.

It learns a single
point wise estimate
of the prediction
and the ability
to learn kind of
this estimate
over all possible
predictions,
rather than just
predicting a single
prediction,
like a point wise
estimate is what
enables us better
generalization
because at test time
what you can do
is instead of just
giving a point wise
prediction estimate,
you can search
over these predictions,
right?

So that's what
you're actually doing
with energy
minimization is you're

searching over the
prediction space
and that's why this
energy landscape is
great.

And then with regards
to your questions
about data,

um,

which data is

important,

uh,

you know,

where can energy

based models

generalize better?

I would say that in

general energy based

models have

generalized better,

which we show as

well as some other

papers show.

I think that on

certain tasks,

energy based models

generalize orders of

magnitude better.

So there's this other

paper that we cite

that shows that for

some synthetic tasks,

energy based models

can generalize like 10

times better than a

feed forward model

because the energy

function or like this

verification is very well defined.

So I can give an example of this, which would be solving a maze.

So if you were to generate a solution to a maze or the feed forward model, you could imagine the maze is like this and solving that.

You need to be able to predict in a single forward pass, all of the different routes or like all of the different turns and everything to solve this maze.

Whereas an energy based model, this energy function will just learn whether or not the solution is a valid solution.

That is the solution verification.

And on that problem, it's much easier to verify than it is to generate.

And because of that, I think you could see that EBMs would generalize much better

because this energy
function is very well
defined.

So I would say that
in general,
EBMs can generalize
better,
but there's certain
problems where they can
generalize much better.

And then with regards
to the question on
what data can EBMs
generalize better on,

I think that in the
short term,

we'll see EBMs
more applicable

for data that is
for applications

where there's limited
data,

particularly because of
this data efficiency
that we've seen with
EBMs.

And you can train them.

They do require more
flops,

at least in the short
term,

but you can train them
such that they kind of
learn this energy
landscape over this
very limited amount
of data.

And because you might

have a limited amount
of data,
you have tons of compute
to work with that data.
And that essentially
means that you can
generalize better
out of distribution
by kind of learning
this huge distribution
over all possible
predictions with this
limited data.

So I see that as the
primary use case in the
short term.

But in the long term,
I think that most data
we could definitely see
EBMs perform better on.
So, yeah.

So in the provocative
opening question,
at first when I heard
it, I was thinking,
oh, this is kind of
like a Pascal's wager
or how many cousins
to save a brother
and those kinds of
questions.

But then it's like,
actually,
we're,
if,
as you're describing,
looking to research
to where the puck is

going and head to a
place,
if we're not already
past it,
where compute isn't
a limitation.
And then it's like,
yes,
it is worth it to
spend more compute.
So that was just,
that was definitely
very counterintuitive,
but also helps
contextualize.
And yet,
out of that
provocation
are coming
models which
could be
more
efficient
per watt
and per
data set.
So it just
is like,
but that
question
is the
entry point
to understanding
what we're
really limited
in.
Okay,
I'll read a

question from
the live chat.

So Tucker

Chambers wrote,

do any

current models

generalize into

the real world

at all?

Maybe Tesla

vision-based models

have a leg or

a wheel up on

the competition

as they're using

real world data

in the physically

embodied sense?

robots?

Yeah,

I think this is

a debatable

question that

depends on who

you ask.

I mean,

I think,

yes,

some models

are generalizing

to the real

world.

The question

is how well

are they

generalizing

and are they

generalizing to

the point
where they're
widely useful?

Robotics is
very much
unsolved and
seems to be
very far from
being solved,
which is one
of the biggest
use cases
of AI.

Completely
autonomous
driving has
been promised
to be solved
for a long
time,
but it's still
unsolved.

We've still,
even with
Waymo,
there are still
people watching.

So I would
say that some
models are
generalizing,
but they are
not generalizing
as well as
humans would,
which is the
core challenge.

Generalization to

the same extent
as humans,
the same
performance.

Ideally,
given the same
data, all of
these models are
trained on orders
of magnitude
thousands of
times the amount
of data that
humans have
ever seen.

So,
yeah,
I hope that
answers your
question.

So they're
generalizing,
but not to the
extent that we
would like.

And then,
Daniel,
I want to
kind of hit
on what you
said earlier,
which is like
this provocative
question of,
is it just a
trade-off?

Or, you know,
in this regime,

is it really even

a trade-off

anymore?

Because, like,

if our GPUs

are just idling,

if we have

10,000 times

the amount of

GPUs, it

might not even

matter, right?

We might really

just say,

okay, we don't

care about

burning 100 times

the amount of

GPUs, we really

just care about

generalization.

So, yeah,

I think it's

interesting, and I

think that, you

know, we're

definitely not

there, especially

in academia right

now, everywhere.

Maybe the

frontier labs are

starting to get

there, and like

people have said,

like OpenAI

released a video

where they talk

about GPT 4.5,
and they basically
say that, and I
actually have this
on here, that
we're more data
bound than we are
compute bound,
which is cool.

So I'm sure that
many of the
frontier labs are
starting to face
this as well.

And I think that
as this happens
more and more,
we'll tend to
see like a shift
more from like
flop scaling and
that being the
predominant thing
that people care
about to like
data efficiency
and generalization.

Because at the
end of the day,
what really matters
is how well does
your model
generalize once
it's gotten to
this point of
kind of fitting
the training set
already.

So, yeah.

Super interesting.

Again, anyone
can write more
questions in the
live chat, but
how about a
little bit on
the almost
developer experience
here?

Like, what was
it like getting
those results?

Was there a lot
of under the
hood and fine
tuning and what
kinds of software
and methods are
useful and what
are they like
today?

And then what
kinds of methods
could unlock the
amount of attention
that's currently
going into
alternative methods
to make use of
what you're
demonstrating in
the paper?

Yeah.

So, in terms of
getting the results,
it was, I would

say, a very
difficult journey,
mostly because
there's not much
practical advice on
how to train
energy-based models
online.

A lot of the
work that's
out online is very
much theoretical
and also hasn't
really been tested
at scale.

So, a lot of
work in this paper
that's kind of not
necessarily super
cool but is
important nonetheless
for practicality
is just determining
what hyperparameters
are necessary,
how to tweak
things so that
things work,
what are kind of
the core challenges
with stability
and what tricks
do you need to
solve them.

We have a lot of
tricks that we
mentioned in the
paper and that

help us with
stability and can
help with
scalability.

But I think that
this is one of the
key challenges
is just kind of
like pretty much
all models now
rely on tricks
like diffusion
models, there's
tons of tricks,
flow models,
there's tons of
tricks.

Even on a
regressive models
at scale,
there's tons of
tricks with
layer norm and
all this stuff.

So, it's all
about really
figuring out like
what are the
right tricks for
these models and
it took a while
and it was a lot
of just like kind
of banging your
head on the wall
and just trying
things out and
seeing what sticks.

But I think
eventually and to
the point where
we are now,
we have hyperparameters
and general tricks
that make things
work more generally
and we know how
to scale these
things.

So, if you told
me to train a 7b
model and you gave
me the compute,
we could.

So, I think that
once we started
figuring out
how to make
things stable
and scalable
and kind of,
you know,
look good
without loss
spikes or anything
like that,
the results
started to look
really great
and that's when
we started to
get like really
great data
efficiency curves
and other stuff
like that and

it was super
rewarding having
worked on just
kind of tuning
these tricks for
a very long time.

And then with
regards to
long-term
adoption and
kind of making
energy-based
models adopted
more widely
as compared to
something like
diffusion models
or pure
feed-forward
autoregressive
models,

I think that
there's a couple
things that need
to happen.

I think one is
like it would be
great if a
foundation model,
energy-based model
would scale.

That way people
can really see
that these do
work out foundation
models scale.

Our results are
just that,

you know,
given extrapolation
that things could
scale well but we
don't actually know
because we haven't
trained them and we
didn't have the
computing resources
to do that.

So, I think kind
of at scale
would be nice.

Another thing
that would be
helpful is just
like more tricks
and tuning.

with diffusion I
like to bring up
this example where
it took lots of
tricks and papers
to kind of figure
out like how do
I sample from
this, how do I
make sampling
faster, how do I
make them efficient
and more practical,
et cetera, et cetera.

I have no doubt
that over the next
year or two lots
of effort will go
into basically just
having tricks that

are better for
these energy-based
models and kind of
making it so they're
faster to train,
faster during
inference, et cetera,
et cetera.

But I think with
those things and
given kind of the
way that compute is
headed, there will
start to be interest
in the next couple
of years.

So I'm very
hopeful.

Yeah.

Tucker asks a
follow-up question
there.

Can EBM be built
and trained on
normal civilian
hardware?

Or I guess just
more generally
like what kind
of hardware
resources within
the von Neumann
paradigm are most
relevant and what
kinds of
unconventional or
thermo-informational
biomimetic computing

could be a little
bit on the horizon
to go even
beyond that?

Yeah.

So I will
clarify that we
focus very much on
energy-based models
that are, you know,
sort of efficient on
von Neumann hardware
here.

So that's like
models with matrix
multiplications using
backpropagation.

There are other
energy-based models
or like, for
example, predictive
coding, learning
algorithm that are
faster to implement
on other hardware
that I'm by no
means an expert.

So these energy-based
models are very
much strong on
modern von Neumann
architectures,
especially GPUs,
because everything
is done with
matrix multiplications
and is learned
end-to-end with

backpropagation,
which we can
compute quickly
on modern
hardware.

So, yeah, you
can definitely
train these
energy-based
transformers on,
you know,
smaller GPUs.

They do take
more memory
than a
feed-forward
model,
but, you
know, most
of our
experiments were
done at very
small scale
on smaller
GPUs, so
you can
definitely do
this on
potentially laptop
GPUs or like
a, you
know, small
scale school
setup that you
have for the
smaller scale
models at least.

So, yeah,

very akin to
modern hardware.

Can we return
to the slide
with the
architectural
layouts?

Yes, of
course.

should I
just do
this?

Okay.

Yeah.

So, maybe
just highlight
what is
different here
structurally,
and then
possibly take
that towards
what would
that look
like in
practice?

Is this
the same
chat GPT
perplexity
interface that's
just doing
better but
giving the
same kind
result?

Is this
like horse

and buggy
versus car?
And we're
talking about
different kinds
of assessments
like what is
happening under
the hood and
structurally?
And then is
that a hot
swap competing
on the same
kinds of
measures and
input-output
structures that
we see today
with all the
zoo of
available models?
or what
other kinds
of harnesses
are needed
or functions
arise?

Yeah.

Yeah, great
question.

So, I'll
start to dive
into the
first part,
which is
like a
general

overview.

So, with
feedforward
models, you'll
get this input
 X , and you'll
predict this
like, just for
autoregressive
modeling, this
 $X_{\hat{T}+1}$,
which is like
the next
token if you're
turning a
language model.

And then you'll
backpropagate.

With an RNN,
you'll do
essentially the
same thing.

With the
diffusion model,
it's a little
bit different
where these
are normally
not trained
autoregressively,
but I kind of
visualize that
here for ease
of visualization.

But what
you'll do is
you will take

as input
prediction of
noise, and
then you will
essentially predict
like this amount
of noise.

So, you'll
start with
random noise,
and you'll
predict based
off the
schedule and
like the
current time
of the score
here, or
like kind of
denoising to
apply to this
 $\hat{x}_t$
plus one.

And you'll
do this for
all the
different $\hat{x}_t$
plus ones that
kind of go
from like pure
random noise
to the actual
data itself.

With an
energy-based model,
it's a lot like a
diffusion model that

you initialize
this $\hat{x}$ of t
plus one as,
or this next
token prediction
for language models
as random noise
or a random
distribution in this
case.

And then instead
of just predicting
the noise and
like subtracting
out the noise,
what you'll do
is you'll predict
the energy,
and then you
will get the
gradient from
this energy to
this prediction,
right?

And you'll
subtract that
gradient out,
so it's just
like gradient
descent,
and like you'll
multiply by step
size, etc.,
etc.

And then you
will pass this
back into the
model, get the

energy again,
update it using
the gradient,
etc., etc.

And you continue
doing this for
every next token
at once.

So it's worth
noting that we
train these as
like GPT-2
decoder style
language models.

So every single
prediction is made
at the exact same
time.

And that's part of
a lot of the
effort that went
into these like
decoder-only
energy-based
transformers.

And yeah,
you're basically
just predicting
the gradient of
every single
next token
prediction at
the exact same
time using the
energy scaler
for each token.

Yeah, and then
in terms of

kind of integrating
these things from
a more applied
perspective on
actual like
applications or
in a UI, I
see in the short
term very similar
applications to
like what we
currently see such
as like chat
GPT or
perplexity, where
you're just doing
inference with the
language model.
I think that the
in the short term
you'll especially
see like hybrid or
ensembling of like a
feed-forward model
with an energy-based
model, where what
you do is you
initialize the
prediction with this
feed-forward model,
like initialize the
distribution, and
then you pass this
into the energy-based
model and just
refine this
distribution, right?
So you're kind of

just like amortizing
a lot of the work
that's done with
this energy-based
model with this
feed-forward model,
or you can
consider ensembling
these models.

And then another
thing that you
could do is use
like speculative
decoding to kind
of like speed up
the iterative
decoding with these
energy-based
transformers.

So instead of
doing every single
next token prediction
with these energy-based
transformers, have a
lighter weight model,
and then just check
which logits are
kind of, or sorry,
which distributions
are different and
sample that way.

So you kind of don't
have as much
inference costs because
they do require more
flops during inference.

And then in the long
term, yeah, I think

that there's a couple of things that you could potentially see with energy-based models that you wouldn't be able to see with these other feed-forward models.

One thing that's exciting is like compositionality.

So there's been a lot of work with energy-based models and compositionality, basically like combining different models with different attributes and generalizing to something new.

So this could be something like, for example, generating new images or generating new languages or whatever it is you want to do.

That's one thing that you could see in terms of applications.

But I think in the short term, from a practical perspective, most of what you will see would just be similar to what you would see with the

feed-forward model.

It might also be that the way that you do inference with these energy-based transformers kind of gives you like a separate way of reasoning.

Where you're kind of reasoning through this iterative optimization rather than in token space with next token prediction.

So the current models are doing this like next token prediction iterative reasoning rather than doing it like in a latent space.

So, yeah.

What exactly makes a transformer a transformer such that it could be feed-forward, diffusive, or energy-based?

And is that part more durable or could something else be there in that green box?

Yes.

Yeah, great question.

So, I think the definition of a transformer depends on who you ask.

I would say to me the general definition is like you have per token level computation and like global sequence mixing via attention. so every token is operated on independently besides for attention where you kind of mix information globally across the sequence.

And then is there different architectures you can use?

Yes, actually.

I think that we just focus on transformers because it's one of the most popular architectures that has proven itself to be very successful across wide modalities and domains, et cetera, et cetera.

Ideally, you know, you could use any new architecture with energy-based models that comes out.

So there's like a recent paper on autoregressive units or like I think they're also called HNets that is like essentially a new

architecture for
autoregressive modeling.

And I think it'd be
great to combine
these things as well
as long as these
other architectures
are stable and
scalable and
parallelizable in
addition to
transformers.

So I don't think
that energy-based
models necessarily
need to be paired
with transformers.

It's just there's
good synergy here
due to the high
successive transformers
and high computational
efficiency and
stability.

Makes sense.

Could you go to
the energy landscape
with the dog
catching the...

Yes.

Okay.

Yeah.

Okay.

Bert Berkers asks,
do EBM get stuck
in local minima
during inference?

Yes.

Yeah.

That is a great question.

So the answer is yes.

And unfortunately, this is not necessarily something that is easy to avoid.

But this isn't a huge problem because in high dimensional spaces, there are less local minima.

So it turns out that there are less local minima and more saddle points.

So it's hard to get stuck in local minima in very high dimensional spaces.

But you are not guaranteed to converge to a global minimum with these energy-based models.

So part of one of the benefits of sampling many, many times is that you can converge to different local minima, one of which may be your global minimum, and then you use this

as your response.

So I think it would be interesting to try and kind of enforce some constraints to, actually, I don't think it's possible to enforce constraints to have a single global minima because then you would restrict the expressivity of your model and you wouldn't be able to kind of fit very high dimensional data.

But it's very, it's a difficult problem.

And there's lots of theoretical work broadly on MCMC and kind of converging to the best minima.

So, yeah.

Definitely.

There's problems slash datasets that are easy and monotonic, and there's ones that are wickedly designed.

You brought up MCMC and you mentioned sampling as well.

So how do you see sampling-based approaches with variational, optimization-based, incremental approaches?

Where do sampling,

particle-based,
distribution-based methods
come into play?

How do we know where to
use different balances of
those?

Yeah.

I'm not an expert on this
upper face with that,
but I think that all of
these are pretty, like,
disjoint from this work
and not disjoint, I
should say, like,
compatible with this work
and sort of adjacent slash
like, not necessarily in
conflict to.

So you can, in principle,
use particle-based
approaches or other more
complex MCMC samplers to
sample from your EBM.

Like, one example would be
using HMC, Hamiltonian,
Monte Carlo to kind of
sample better.

We just use first-order
information or, like,
Langevin dynamics.

But you could, in principle,
use more complex samplers
as well as, like,
particle-based algorithms
to sample better.

You can kind of consider
that what we are doing,
I'm forgetting the name,

by sampling many times
and choosing the minimum
energy is sort of,
it's like, there's some
reference to this in
statistical physics.

You can also use many of
the ideas from statistical
physics and kind of, like,
inspiring better inference
or sampling algorithms
where you, like,
anneal the step size
or you, like, determine
whether or not the energy
decreases and then if it
increased, like,
decreased the step size
so you kind of better
get to local minima.

There's tons of ways
you can do it.

And I think that broadly,
you know, this work
and the work on MCMC
can only help each other,
basically.

So they're sort of adjacent
and can help in tandem.

Yeah.

Yeah, I agree with that.

There's work on
Stein mixture models
and particle transport
and accelerated optimization
on different kinds
of information geometries.

It's kind of just, like,

exciting how it's all
coming together.

So one reflection,
which I'd be curious
to hear your thoughts
seeing from an academia,
researchy,
and from an industry side,
is, like,
in the data-driven
or data-based approach,
there is some
extant data.

Maybe you can acquire more,
but it's about doing
something
with that data.

Whereas on,
at least in the
active inference,
computational psychiatry,
cognitive modeling space,
often it's more like
a priori systems modeling,
where we're designing
that joint generative model,
but an empirical data set
doesn't actually have
to come into play.

You might be interested
in just
ab initio modeling
of a cognitive agent
and generating synthetic data.

And then there's methods
for tuning the parameters
to recapitulate
empirical parameters,

but it's just sort of
interesting to see
the difference between
having an extant data set
as, like,
the block of marble
that you want to do
the best on.

Like,
you have a language corpus
and you want it to be
fluent and write
awesome PhD dissertations,
but not much
structural information
about linguistics
has to be encoded.

And that's the sort of
stochastic parent-type
critique perspective
in comparison
or in complement
to making models
models
who structurally embody
linguistic constraints
and using that
to then meet
in the middle,
also you'd hope,
with reasonable-looking
linguistic expression,
but that approach
wouldn't necessarily
need to be trained
from unstructured
text corpuses.

So if we're thinking

about general-purpose

EBMs

or domain-specific

EBMs,

arbitrarily narrow

to arbitrarily broad,

where is that

going to be

from just

dumping a blob

of data,

setting the number

of parameters

and some other

aspects,

kind of like how

vision neural networks

might work,

just,

here's my data folder,

here's the architecture,

now go,

and where is it

going to take

more of a

scaffolded,

structured approach

that

has a model

that embodies

aspects of the

situation.

Yeah.

Yeah,

that is

very hard.

I think that

it's very dependent

on the amount
of data that you
have and the
problem.

I think there
are certain
problems where
you need a
prior and that
without this
prior you sort
of,

or some inductive
bias with your
model,
to be more
data efficient,
such that without
this inductive
bias you will
not succeed.

And I think
that particularly
we'll see that
more often on
domains with
limited data.

Right?

So if you have
very limited
data,
having just a
model and just
the data,
no prior on the
structure or
inductive biases
of your model,

your model will
very much overfit
that data and
not generalize.

I think the
reason that we're
able to do this
with language and
video and audio
is because we
have huge amounts
of data.

I mean,
data that is
on orders of
magnitude more
than any human
will ever read
or see in
their lifetime.

So it may
very well be
that when we
actually care
about,
well,
we do care
about this,
but when we
care very much
about achieving
human level
intelligence,
given the
same amount
of data as
humans,
we need

stronger priors
on architecture
and structure,
et cetera.

It may also be
that from a
practical perspective,
these are more
useful on
specific modalities
or domains
where we don't
have as much
data as we do
for language
or for video.

So for example,
for medical data,
I am not an
expert on this,
but I very much
believe that there
is many more
data constraints
with medical
data than there
is with just
general videos
online or general
language online.

And because of
this,
it may be that
inductive priors
or kind of
structures in
your model that
are more akin

to how human
cognition works
or however your
model of psychology
works are much
better at making
decisions and
performing or
generalizing.

so I think
the key is
really data.

You know,
if you have
infinite data,
you don't need
any priors.

You just learn
everything from
the data.

If you don't
have infinite
data, then
priors are helpful.

And this is
kind of broadly
what we've
observed in AI
in the last
decade where
over time,
the amount of
priors that these
models have has
decreased and
you really just
like let the
model fit the

data.

You don't really

make any

assumptions.

10 years ago

or 15 years

ago and we

didn't have that

much data in

compute, people

did a lot of

hand-tweet things

and engineering

and feature

engineering, etc.

etc. for their

vision models to

make them work

better.

And they did

work better with

those hand-engineered

features at the

time.

So I think the

key is like how

much of a

bottleneck is your

data?

I believe I

resonate with you

on that with

this sort of

zigzagging,

forward-moving

pendulum.

between expert

systems-focused,

so-called, and
more unstructured.

And to combine
the structured and
the unstructured is
the chocolate chip
cookie that has
these kind of
neurosymbolic,
hybrid, synthetic
intelligence.

There's just a
million ways to
point at these
architectures that
have different
sorts of elements
ranging from
logical,
discrete, to
interpolated,
continuous.

So that's
awesome.

Okay, last
sort of two
areas that I'd
love to hear your
take on, and
then I'll see if
there's any final
comment in the
live chat, would
be how would
somebody who's
wanting to learn
and apply get
onboarded?

What resources did
you all provide in
the repository?
and what kinds of
things would
somebody want to
learn up on and
train up on to
be able to
replicate and
extend?

And then, of
course, where is
this all going?

What is the next
shoe to drop?

What is the next
pheromone that will
be deposited?

Yeah, I like that
last question a lot.

With the first
question, yeah, we
tried to make it a
little bit easier to
get started.

So, at the end
of the paper, like
section, I think,
H and I, there's
some intro to
EBM tutorial and
stuff like that and
some pseudocode.

There's also some
videos by Yannick
Kilcher.

He's a YouTuber

that he made on
energy-based models
that I think are
really great for
getting started.

But I recommend
checking out those
videos and checking
out those later
sections in the
paper that are
sort of more
easily written.

Because I found
it, even for me,
you know, like I've
been doing EBMs
for a while, very
difficult to read
a lot of the
literature because
it's super
theoretical and
sort of like maybe
not catered towards
beginners.

So, I think that,
you know, just
maybe checking this
out or checking out
some intro or
tutorials on EBMs
would be great, as
well as just PyTorch
in general.

Yeah, the Yannick
Kilcher videos are
great as well.

And then the second question, where are we headed?

Yeah, that is a great question.

I think in the long term, like what we'll see with diffusion models is kind of like what we saw with energy-based models is kind of like what we saw with diffusion models, where like at first they're kind of like, oh, these are impractical, no one's going to use these, to like, okay, maybe there's some promise, to like, okay, these have narrow applications, to like, oh, wow, we can apply these to almost everything.

Like now, diffusion language models are very much becoming very popular when it used to just be image generation.

And now they're used for planning and robotics, et cetera,

et cetera.

I think we'll see the same thing with energy-based models, where over time they will become more and more widespread because of, you know, potentially better generalization.

So I think that that is the future.

I think the question is really like how much time, how much compute do we need, and what are the kind of core tricks and algorithms, you know, and the improvements that can be made.

So that is what I see.

But I think that if you believe that generalization will be the blocker in the next decade, for sure energy-based models are a good way, good bet to put your money on.

So.

If you generalize from your past experience to update your belief that

generalization is
important belief to
have updated to,
this is an
interesting architecture.
adventure.

Yeah.

I'm trying to call you
along there, but
yeah.

Yeah, yeah, yeah,
yeah.

I agree with that.

Any final comments or
just what's next for
you and your graduate
and beyond adventures?

Not much final
comments.

Just thank you guys
for the interest and
it's been kind of
amazing.

I did not expect as
many people to be
interested in the
paper.

So it's super,
super grateful for
people even listening
and for giving me the
time to speak.

In terms of my
future, yeah, we'll
have to see.

It's hard to know.

I mean, I definitely
plan to continue

working on EBMs, but
yeah, I have some
crazy ideas planned in
the future as well.

So we'll see how
those go.

Amazing.

Thank you.

Super informative.

There's really a
big community of
researchers and people
kind of in the
engine room.

And I think also
broader and broader
peripheries of
interest from
everything AI ethics
and just the future
of what is going to
happen.

So it's awesome to
see.

And I hope we can
have a follow-up.

I hope so as well.

Yeah.

Thank you so much for
having me.

Really appreciate it.

All right.

Peace.

Thank you.

Bye.

Bye.

Bye.